Forkable sandboxes let computers explore alternatives without doing work twice.

Self-organization → self-driving cars
→ self-driving compute.

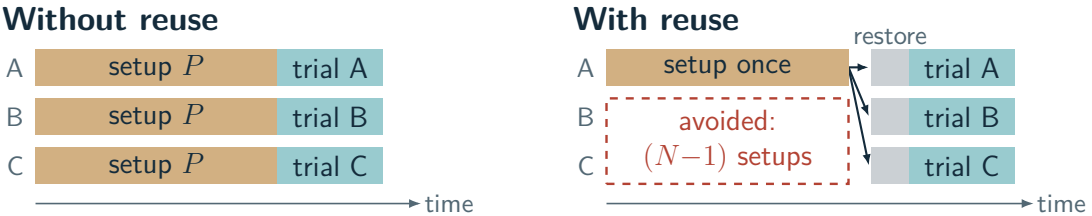


Compute once. Count once. Publish once.

Yossi Eliaz
Principal Engineer, Incredibuild
Associate Professor, HIT

Cambridge Systems Research Group, 15 October 2026. Employer affiliation disclosed.
Tags: PUBLISHED published work MEASURED our measurements BUILT our code
DESIGN proposed, not run ILLUSTRATION worked example

In parallel computing, the costly work is the work done twice.



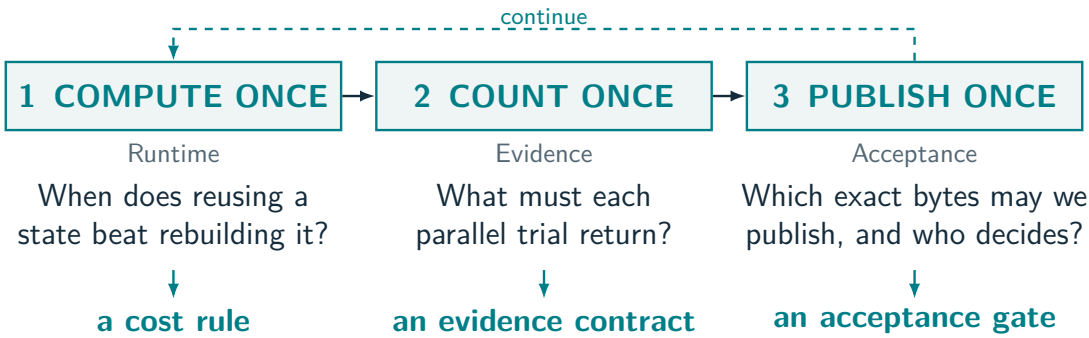
Cache reuse a previous run's output.

Fork reuse a reached state.

Sandbox keep each trial's writes private.

Schematic, one time scale in both panels. Restoring is not free; the runtime section prices it.

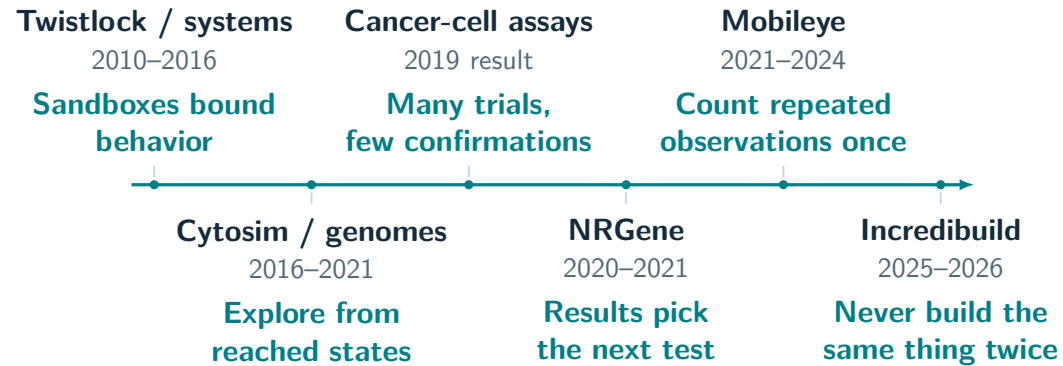
Sharing work creates three rules.



You will leave with all three.

The talk takes the rules in order. This map returns after the examples and at the end.

Each stop on my path supplied one piece of this system.



Career periods; cancer-drug result (2019). Periods overlap; not to scale. Lessons are my reading, not results.

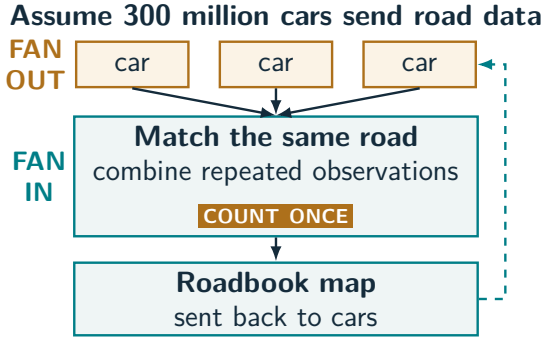
PUBLISHED DESIGN

A car fleet fans out, then fans in to one map.



Mobileye autonomous test vehicle,
Manhattan, June 2021.

My work, 2021–2024:
junction perception.

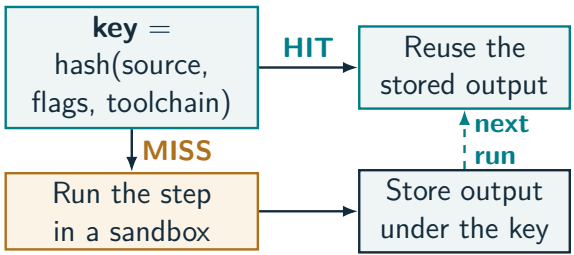


Self-driving compute: many runs from one state, one combined result.

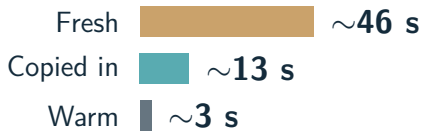
300M fleet: speaker-reported. Anonymized data flow; Roadbook workflow. Photo: Mobileye press kit.

PUBLISHED MEASURED

A cache skips work that a previous run did.



TOKIO COMPILE



Copied in: an earlier runner's build files, not its running state.

The sandbox hides undeclared files,
so a step cannot read what the key omits.

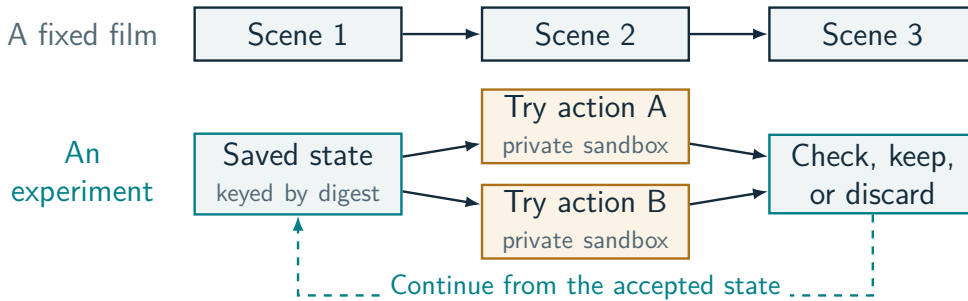
Distributed builds rely on it: one machine reuses another's output.

Tokio: Eliaz, issue #8200, 8 June 2026; author-reported external Incredibuild demo, approximate compile times.

A fork extends caching from files to a reached execution state.

Cache: one step's output. **Fork:** the state after many steps.

Installed dependencies, a reproduced failure, a warmed process.



Organizing analogy and proposed execution model. Stored-program hardware remains the substrate.

The forkable API prepares once and runs alternatives in private children.

Same three edits, same test, same selection rule.

Python loop

```
out = []
for edit in edits:
    env = prepare()
    env.apply(edit)
    out.append(test(env))
best = select(out)
```

`prepare()` **3** times.

Forkable API

```
parent = checkpoint(prepare())
children = fork(parent, n=3)
execute(children, edits)
out = evaluate(children, test)
best = select(out)
```

`prepare()` **once**; **3** private children.

Proposed API sketch, not an existing SDK. Both columns are Python; each check returns its artifact and evidence.

PUBLISHED

Caches, forks and sandboxes are old; coupling them to evidence and acceptance is new.

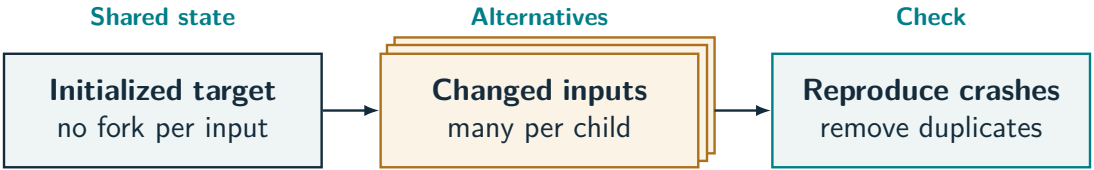
	Reuses or protects	Must guarantee	Known systems
Cache	a run's output	key covers all inputs	ccache, Bazel remote cache, Incredibuild
Fork	a reached state	capture keeps what the next step needs	SnowFlock on Xen (2009), CRIU, Firecracker snapshots
Sandbox	reads and writes	undeclared inputs hidden; writes private	containers, gVisor, microVMs

New: count shared evidence once; publish checked bytes once.

Not claimed today: a fork speedup (our agent runs were serial), or live-memory restore.

SnowFlock: Lagar-Cavilla et al., EuroSys 2009. Firecracker: Agache et al., NSDI 2020. CRIU, gVisor, ccache, Bazel.

Reuse makes fuzzing fast; only checks turn crashes into findings.



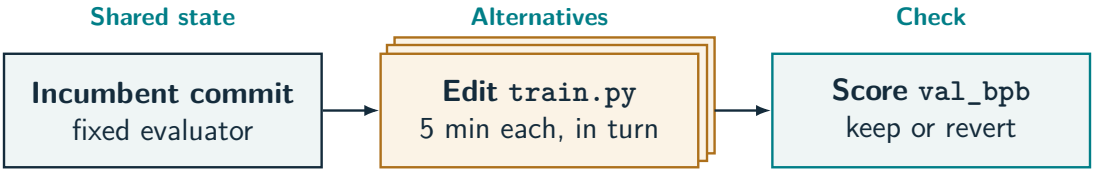
AFL++: **10–20**× faster; CyberGym: **56** crashes → **22** confirmed

CyberGym, a separate study: OpenHands + GPT-5 wrote and ran inputs on 431 projects.

Set up the target once; count each bug once. COMPUTE ONCE COUNT ONCE

AFL++ persistent-mode docs, typical gain vs a fork per input; CyberGym report, latest-code GPT-5 campaign.

Autoresearch kept 23 of 126 attempts against a fixed evaluator.



126 attempts, **23** kept: **0.997900** → **0.969686**, **2.8%** lower

AlphaEvolve: a deployed Borg heuristic recovered 0.7% of Google's compute, on average.

Reuse the incumbent; the evaluator promotes. COMPUTE ONCE PUBLISH ONCE

Karpathy, autoresearch protocol; reported session. Google DeepMind, AlphaEvolve (2025).

250 samples raised SWE-bench Lite coverage from 15.9% to 56%.



1 sample: **15.9%** → **250** samples: **56%** of tasks

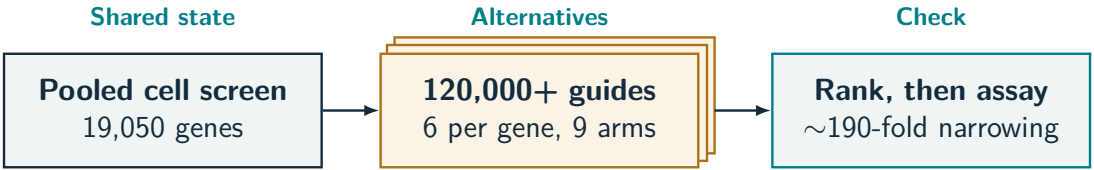
Coverage: tasks with at least one correct patch (DeepSeek-Coder-V2-Instruct).

Coverage is not selection: one patch must be picked. PUBLISH ONCE

Brown et al., Large Language Monkeys (2024); SWE-bench Lite, pass at least once.

 PUBLISHED

Our CRISPR screen narrowed 19,050 genes to about 100 candidates.



MCF-7 assays: **10 of 13** targets, **6 of 8** drug combinations

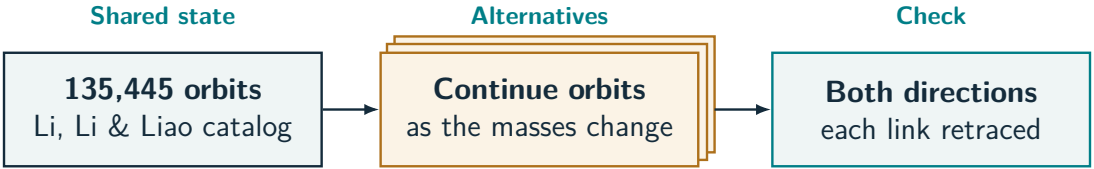
Targets increased SI-12 sensitivity; combinations improved cell killing in vitro.

Six guides are six looks at one gene: count the gene once. COUNT ONCE

Gilad, Eliaz et al., Communications Biology 4:399 (2021), Fig. 3. Ranking nominates candidates for validation.

MEASURED

With Ori Chamo, we connected two apparent branches of one orbit family.



26 of 26 selected connections hold in both directions

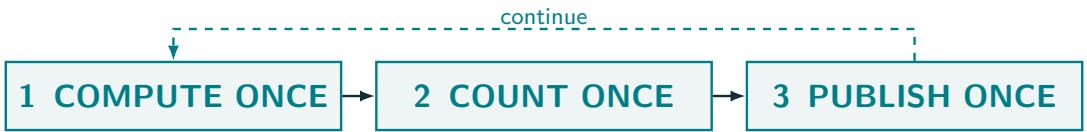
One sampled solution component; 2 stability changes re-checked at 60 digits.

Two branches in a plot, one family in solution space.

COUNT ONCE

With **Ori Chamo**. Chamo–Eliaz preprint (2026), not peer reviewed; Atlas numerical records.

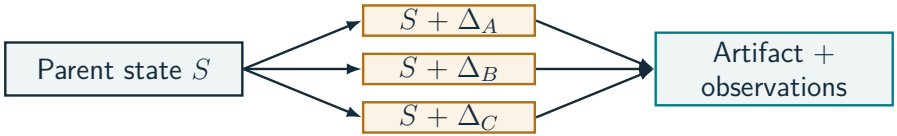
Five fields, one loop, three rules.



Compute once. Count once. Publish once.

The five previous examples, by the rule each one shows. None measures our integrated runtime.

The next step decides which state to reuse.



Next step	Reuse by	Supplies
Edit or rebuild	**Cache**	files, cached outputs
Continue a live run	**Restore** a checkpoint	the memory it keeps
Reuse an answer	**Replay**	the recorded response

MEASURED **Restore fan-out** on islo: 255/256 restore–run–capture calls succeeded.

DESIGN Does live memory survive our restore? Untested until Gate 0 runs.

Choose the cheapest mechanism that supplies what the next step reads.

CRIU, gVisor, Firecracker, rr; SnowFlock (EuroSys 2009); DSec. Restores: arXiv:2607.09689v4, Table 2.

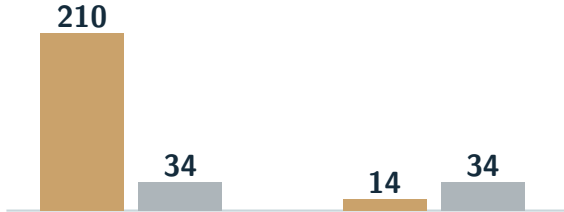
A restore pays only if it beats a warm cache.

THE RULE

$$(N-1)P > H+N(R+D)$$

N	alternative runs (8)
P	setup each run would repeat
H	save the state once (2)
$R+D$	restart + private writes (1+3)

AVOIDED SETUP vs OVERHEAD

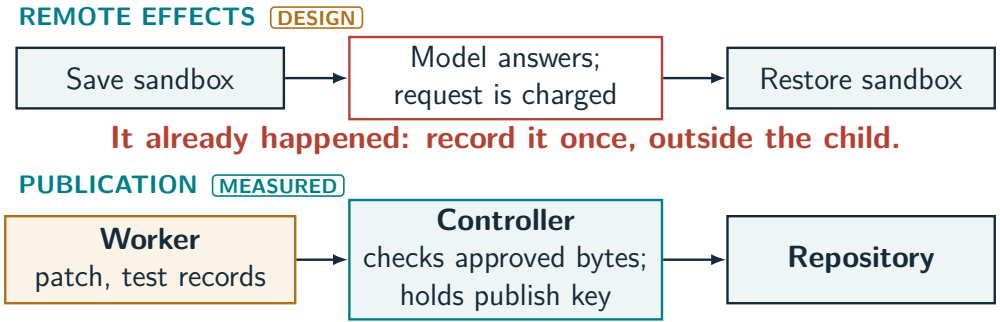


Fastest public start: **73 ms** median to a first command, so P must be real preparation.

Price a restore against a warm cache, not a cold start.

Illustrative additive model, resource-seconds. Start: ComputeSDK Burst TTI (Isorun median), 2 October 2026.

A restore cannot undo remote effects, and a fork must not hold the right to publish.



Record effects once. Keep the publish key outside every fork.

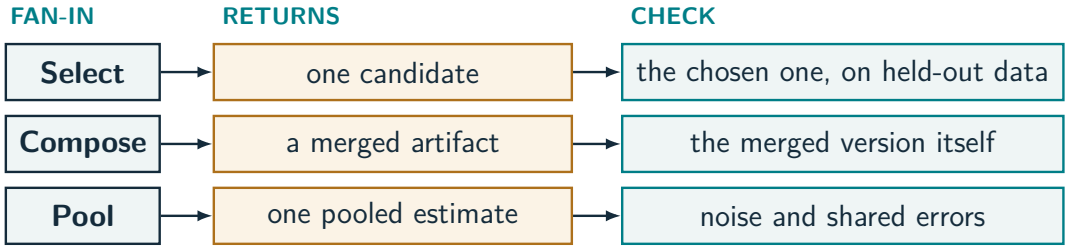
Output commit: Speculator, external synchrony, Remus. Effect journal: proposed; publish path: our factory runs.

A receipt is evidence's cache entry, written outside the child.

A RECEIPT SHOULD RECORD (DESIGN)		OUR FACTORY LOGS FOUND (MEASURED)
Check	what ran + outcome	"CI passed" in 3 s; evaluation skipped
Artifact	exact bytes + hash	Code hash recorded; source unavailable
Actor	human, harness or service	"Human approved"; automation answered
Inputs	observation IDs	→ Lets a shared observation count once

Three audit findings from our factory records (SELFHOST-2 approvals and adapter; CI, 28 September 2026). Typed receipt fields are proposed. Related provenance: PASS, CamFlow, in-toto.

Each fan-in operation needs its own check.



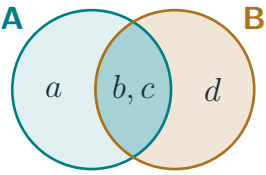
Why test the merge? Each change starts a 1 GB worker; the server has 2 GB.

A+B ✓ | A+C ✓ | B+C ✓ | A+B+C = 3 GB ×

Illustrative capacity example, not a recorded incident. Union and intersection are further fan-in rules (Q&A).

Overlapping trials add less than their count.

SAME OBSERVATIONS: COUNT ONCE



4 distinct
2 shared
B adds **1**

SHARED ERRORS: COUNT FOR LESS

100 estimates, error correlation $\rho = 0.1$

$$N_{\text{eff}} = \frac{100}{1 + 99(0.1)} = \mathbf{9.17}$$

Reduces noise like about 9 independent ones.

Seen before: one road, many drives (Roadbook); one gene, six guides (screen).

BUILT Our reducer rejects duplicate observation IDs and carries lineage.

Proposed: covariance-aware weights; refuse unsupported precision claims.

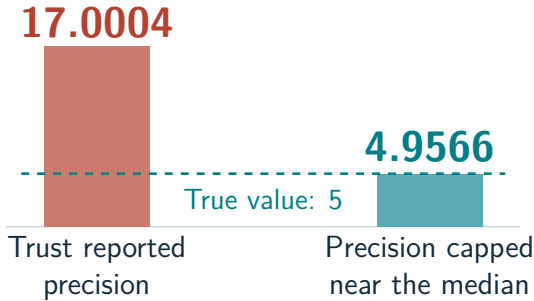
Illustrative: exact set IDs; equal noise, common pairwise correlation. Reference reducer; arXiv:2607.09689v4.

A worker that overstates its precision can take over the average.

$$\hat{\mu} = \frac{\sum_i p_i \, x_i}{\sum_i p_i}$$

p_i: **reported** precision (1 / variance).
More precision, more weight.

Synthetic: one worker, 2,000 points near 17, information per point $\times 50$.



Check outside the child, including its error bars.

Reference demo: one synthetic stress case, not a Byzantine guarantee; the rule needs trusted precision.

Accept only bytes whose promise was tested.

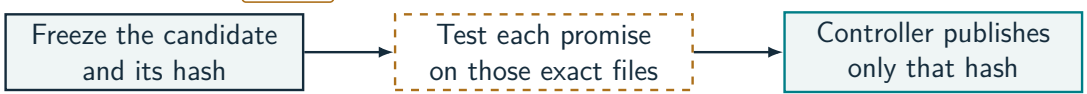
OUR FACTORY RUN MEASURED serial, Docker sandboxes, nothing forked



✓ Suite after each repair: 1,972 cases (incl. skips), 0 failures.

Review approved. Suite green. The new code path had no direct test.

PROPOSED GATE DESIGN



a hash proves identity, not correctness

BUILT hash-bound publish

SELFHOST-2, 27 September 2026: PR #2351, zozo123/ariflow-swfactory; merged 3 October 2026 after I reverted the out-of-scope, untested path outside the factory. Gate: proposed; hash-bound publish: built.

Save first; let one controller publish once.

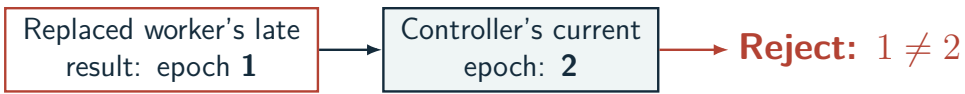
OUR SECOND FACTORY RUN MEASURED

Delivery refused three times; then cleanup **erased an approved patch**.

1 SAVE BEFORE CLEANUP DESIGN



2 REJECT STALE RESULTS DESIGN



Proposed: an epoch fence, and a publish keyed by the digest.

Second factory run: observed. Save order, epoch fence: proposed. Prior art: leases (1989), Chubby sequencers (2006).

Five questions remain open; the preregistered tests have not started.

COMPUTE ONCE

1	Does the snapshot we restore keep live memory?	**Gate 0:** RAM-only nonce
2	Does restoring beat a warm cache?	**H1:** N = 3, 6, 12 sandboxes

COUNT ONCE

3	Do restored siblings fail together more?	**H2:** race-test co-failure
4	When should the reducer refuse a precision claim?	**Design:** dependence, calibration

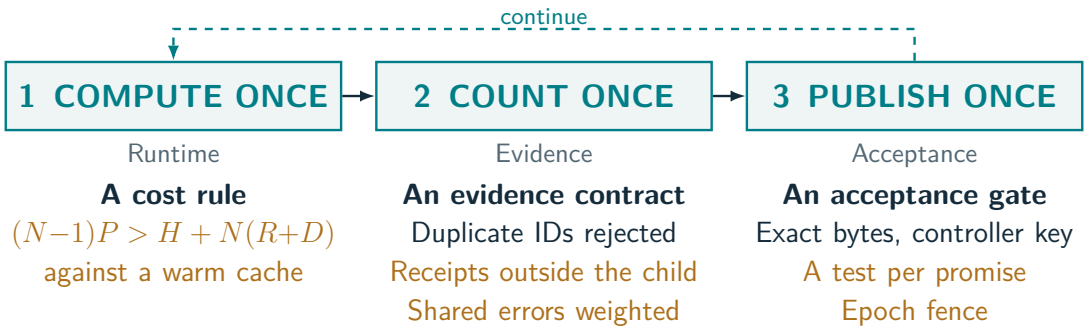
PUBLISH ONCE

5	Does the epoch fence hold across restores?	**Design:** TLA+, no TLC run

Until Gate 0 runs: many restores of one snapshot, not a memory fork.

PREREGISTRATION.md, tag prereg-v1 (amended to v3; no rule changed). Q5: TLA+ design, no TLC run.

Contributions: three rules for doing work once.



Compute once. Count once. Publish once.

Built: publish check, duplicate-ID reducer. Measured: serial factory runs, nothing forked. Gold: proposed or modelled.